

CS-600: Software Design and Development

1-2 Short Paper: Principles of Effective Design

Malgorzata Debska

Southern New Hampshire University

Matthew Scuteri

April 21, 2026

Client Requirements Analysis and Development Paradigm Selection

In software development, clearly understanding both functional and nonfunctional requirements is essential for building successful applications. While functional requirements define what a system should do, nonfunctional requirements shape how well the system performs and how users experience it (Sommerville, 2016). This paper reviews the provided client requirements, finds missing nonfunctional requirements, analyzes how these requirements influence project scope, and recommends a proper development paradigm.

Summary of Functional Requirements

The client's functional requirements describe a system designed to manage client records. The system must store key client information, including name, address, contact details, and phone numbers. Also, users must be able to search for records using either a single attribute or a combination of attributes, allowing for flexible data retrieval. The application must also support full data manipulation capabilities, including creating, updating, and dropping records. Role-based access control is needed, with various levels of permissions such as read-only users and administrative users who can change or delete records. Finally, the system must confirm all input data fields to ensure accuracy and consistency. Overall, these requirements define a structured

data management system focused on usability, access control, and data integrity (Pressman & Maxim, 2020).

Additional Nonfunctional Requirements

While the functional requirements outline what the system should do, several important nonfunctional requirements (NFRs) are missing and should be included. First, performance requirements are critical. The system should define acceptable response times for search queries and data updates to ensure efficient user interactions. Without this, users may experience delays that reduce productivity (Sommerville, 2016). Second, security requirements are essential due to the sensitive nature of client data. These should include encryption, secure authentication, and protection against unauthorized access. This is especially important because role-based access alone does not guarantee full data protection (Pressman & Maxim, 2020). Third, availability and reliability must be considered. The system should ensure high uptime and include backup and recovery mechanisms to prevent data loss. Users depend on consistent access to client records for daily operations. Fourth, scalability should be addressed. As the number of client records grows, the system must support performance without degradation. Finally, usability and maintainability are important. The interface should be intuitive for users, and the system should be designed in a way that allows developers to easily update and extend functionality over time. These non-functional requirements are crucial because they directly affect user satisfaction and system effectiveness. Ignoring them can lead to system failure even if all functional features are implemented correctly (Sommerville, 2016).

Impact on Scope and Priorities

The combination of functional and nonfunctional requirements defines the overall scope and priorities of the project. Functional requirements prove core capabilities, such as managing and searching for client data. However, nonfunctional requirements influence how these features are implemented and prioritized. For example, performance requirements may require optimized database queries and indexing strategies. Security requirements may introduce more complexity, such as authentication systems and encryption protocols. Similarly, scalability considerations may influence the choice of architecture, such as whether to use cloud-based solutions (Pressman & Maxim, 2020). As a result, non-functional requirements often increase the complexity of the project and require added planning and resources. They also help prioritize development efforts by ensuring that critical system qualities such as reliability and security are addressed early in the design process.

Recommended Development Paradigm

Based on the identified requirements, an Agile development paradigm is the most proper approach for this project. Agile emphasizes iterative development, continuous feedback, and flexibility, which are essential when both functional and nonfunctional requirements may evolve over time (Pressman & Maxim, 2020). Agile aligns well with this project because it allows the development team to deliver core functionality early, such as basic client record management, and then incrementally add features like advanced search, role-based access, and security enhancements. This approach also supports ongoing refinement of nonfunctional requirements, which are often clarified as the system is tested and used. Also, Agile promotes collaboration between stakeholders and developers, helping to reduce misunderstandings in requirements as a

common cause of project failure. Modern architectural thinking also supports flexible approaches, as different paradigms can be combined depending on system needs (Chandrasekaran & Krishnan, 2022).

Application to Professional Practice

Analyzing requirements and selecting a suitable development paradigm are essential skills in professional software development. In real-world projects, incomplete or unclear requirements are common, and developers must find gaps and propose solutions. This process ensures that both functional and nonfunctional aspects of a system are addressed, leading to higher-quality software. It also helps teams choose development approaches that align with project goals, timelines, and complexity (Sommerville, 2016). By carefully evaluating requirements and selecting the right paradigm, software professionals can reduce risks, improve system performance, and deliver solutions that meet user needs effectively.

Conclusion

In conclusion, while the client's functional requirements provide a solid foundation for the application, other nonfunctional requirements are necessary to ensure system quality and usability. These requirements significantly influence project scope, priorities, and design decisions. By adopting an Agile development paradigm, the team can effectively address both functional and nonfunctional needs while still being flexible and responsive to change. This structured approach increases the likelihood of delivering a successful and reliable software system.

References:

Sommerville, I. (2016). *Software engineering* (10th ed.). Pearson.

Pressman, R. S., & Maxim, B. R. (2020). *Software engineering: A practitioner's approach* (9th ed.). McGraw-Hill.

Chandrasekaran, P., & Krishnan, K. (2022). *Domain-driven design with Java: A practitioner's guide*. Packt Publishing.